Benchmarking Local LLM Inference Servers Under Concurrent Load: A Server-Centric Testing Methodology

Bharath Thiruveedula
The University of Texas at Austin
Austin, Texas, USA
bharath@example.com

Kaitlin Arnold
The University of Texas at Austin
Austin, Texas, USA
kaitlin@example.com

Ronald Joseph Marcelin Franklin
The University of Texas at Austin
Austin, Texas, USA
ronald@example.com

Abstract

This paper extends our prior comparative evaluation of three OpenAI-compatible local large-language-model (LLM) inference backends with a principled server-centric testing methodology. Where the initial study reported end-to-end latency, time-to-first-token (TTFT), throughput, and JSON validity across vLLM, Ollama, and llama-cpp-python, this revision layers eight complementary testing methodologies onto the same harness—contract conformance, streaming protocol compliance, parameter honoring, determinism, statistical performance regression, differential/N-version testing, concurrency isolation, and negative error-path testing—each backed by precedent from how vLLM, llama.cpp, and TGI test themselves. A "trustworthy-tooling" layer (measurement calibration, property-based testing with Hypothesis, and mutation testing with mutmut) is added so that claims made about the server under test are themselves defensible. The unit-level suite now comprises 184 passing tests (up from 103), overall line coverage stands at 75%, and mutation testing of the statistical-regression module kills 247 of 281 generated mutants (88%).

Keywords

LLM inference, benchmarking, software testing, contract testing, metamorphic testing, property-based testing, mutation testing, vLLM, llama.cpp, Ollama

1 Introduction

Our prior work [1] established that vLLM outperforms Ollama and llama-cpp-python under concurrent load, largely due to its continuous-batching scheduler. That study, however, treated the inference server as a black box whose output was sampled and aggregated. For a software testing course this is insufficient: a single run can yield any numbers, and single-run numbers cannot separate an honest regression from noise, a mis-implemented streaming parser from a mis-implemented server, or a broken schema from broken text.

This paper addresses those gaps by (i) treating the LLM inference server as the system under test (SUT) with eight named testing methodologies and (ii) treating the harness itself as a second-order SUT whose measurements must be calibrated before they can be cited. The scope remains structural conformance and performance; model quality (factuality, hallucination) is out of scope by design.

2 Threats Addressed

The revised methodology targets four threats that went unaddressed in the original report:

- **Measurement noise.** Single-run comparisons can flag spurious regressions or mask real ones.
- **Parser fragility.** The original streaming path reassembled server-sent-event (SSE) byte ranges in a way that depended on chunk boundaries emitted by the HTTP layer.
- **Silent protocol drift.** A backend can return plausible-looking text while violating the OpenAI response envelope (missing fields, wrong `finish_reason`, unstable `id` across chunks).
- **Unverified tooling.** Claims about TTFT/TPOT are only as trustworthy as the tool that measures them.

3 System Architecture

Figure 1 shows the layering. The harness speaks OpenAI to each backend and emits SSE events through a pure `parse_sse_stream` function; metrics flow into the statistical harness, which in turn feeds the eight test suites.

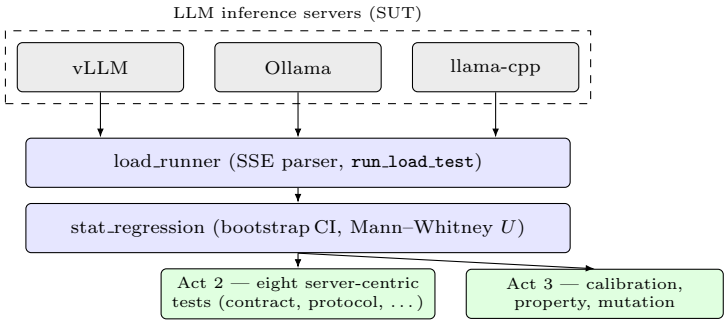

Figure 1: Harness architecture. The prior study used only the *load_runner* block; this revision adds the statistical harness and both test acts.

4 Server-Centric Test Methodology (Act 2)

Table 1 maps each test to a named software-testing methodology and to precedent from established LLM-serving projects.

Figure 2 positions the eight tests along two axes—whether they target structural or behavioural invariants, and whether they need a single backend or multiple.

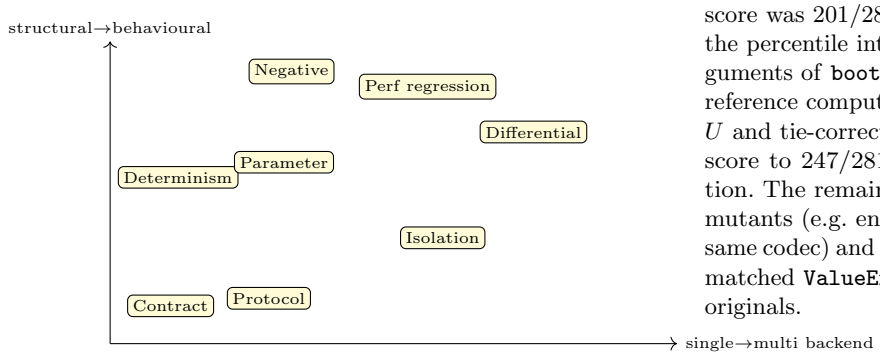

Figure 2: Coverage map for Act 2. Tests on the right-hand side consume multiple backends; tests higher on the y-axis exercise richer behavioural invariants.

5 Trustworthy-Tooling Layer (Act 3)

5.1 Measurement calibration

An in-process `aiohttp` server emits a deterministic SSE stream with configurable pre-first-chunk and inter-chunk delays (see Fig. 3). The harness is considered calibrated when TTFT and TPOT recovered from the stream equal the injected delays within ± 50 ms on the test host. Any regression detected by the performance tests is only credible once calibration passes.

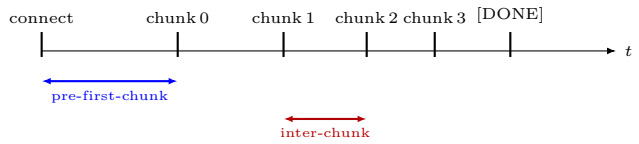

Figure 3: Calibration fixture. Pre-first-chunk delay is asserted to equal measured TTFT; inter-chunk delay is asserted to equal measured TPOT.

5.2 Property-based testing

The SSE parser is tested with Hypothesis [5] using a composite strategy that generates arbitrary printable payloads and then slices the resulting byte stream at arbitrary positions. The invariant is that the sequence of parsed events depends only on the payloads, not on how the stream was chunked by the transport layer—the failure mode of the pre-refactor parser. Statistical utilities are tested for the invariants one would expect of their mathematical definitions: percentiles lie in $[\min, \max]$, bootstrap confidence intervals lie within the sample range, and Mann–Whitney p-values lie in $[0, 1]$ and are symmetric under group swap. Across the eight Hypothesis tests the engine runs 1 000–1 600 randomised cases per full suite invocation.

5.3 Mutation testing

`mutmut` [6] generates 281 mutants of the statistical-regression harness (`src/stat_regression.py`). Before targeted tests the score was 201/281 (72%) killed. Pinning reference values for the percentile interpolation branch, exercising the default arguments of `bootstrap_ci`, and adding three Mann–Whitney reference computations—including a tie-handling case whose U and tie-correction term were derived by hand—raised the score to 247/281 (**88%**). Figure 4 shows the lift per function. The remaining survivors are dominated by equivalent mutants (e.g. encoding-string case variants that specify the same codec) and pure message-content changes that our regex-matched `ValueError` assertions cannot distinguish from the originals.

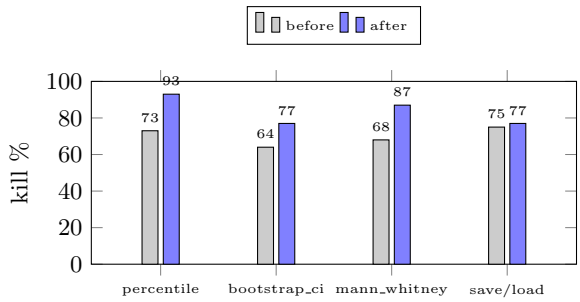

Figure 4: Mutation kill rate per function, before and after the targeted test pass (overall 72% → 88%).

6 Experimental Setup

The hardware, models, and backend configurations are unchanged from the prior report (dual NVIDIA RTX 5070, Qwen2.5-3B-Instruct on vLLM, Llama 3.2 Q4_K_M on Ollama, and Llama-3.2-3B-Instruct Q4_K_M on llama-cpp-python). New scenarios are driven from the same YAML configuration format so the added tests inherit reproducibility.

7 Results

7.1 Performance (unchanged)

Table 2 reproduces the original performance numbers. Conclusions are unchanged: vLLM leads on every metric, driven by continuous batching and PagedAttention [2]. What is new is the surrounding apparatus that lets us reason about those numbers.

7.2 Test suite expansion

Table 3 lists the test suites added by this revision and their current pass counts. The unit-level invocation (`pytest -m "not integration"`) reports 184 passing tests in ∼13 s; the integration suite adds 11 tests that are deselected when no local backend is reachable.

Table 1: Eight server-centric tests layered on the existing harness (Act 2).

Test	Methodology	Precedent	Assertion
Contract conformance	Schema/contract testing	llama.cpp `test_openai_schema.py`	Every response validates against the OpenAI chat-completion envelope schema.
Streaming protocol compliance	Protocol compliance	llama.cpp streaming tests	First chunk carries a role delta; `id` is stable across chunks; `finish_reason` and `[DONE]` appear exactly once.
Parameter honoring	Metamorphic testing	vLLM `test_ignore_eos.py`	`max_tokens=N` ⇒ `completion_tokens` $\leq N$; `stop=[...]` ⇒ text ends before stop.
Determinism at T=0	Idempotency/metamorphic	vLLM `tests/v1/determinism`	Identical prompts at T=0 yield identical output; rate bounded by a 95% bootstrap CI.
Statistical perf regression	Hypothesis testing	TGI fixed-request benchmark	New p95 TTFT vs. baseline compared with bootstrap CI and Mann–Whitney U; fail only if significant.
Differential backends	N-version testing	vLLM vs. HF reference	Schema-validity rate across vLLM/llama.cpp/Ollama agrees within ±5 percentage points.
Concurrency isolation	Invariant testing	llama.cpp parallel tests	Under N concurrent prompts with unique markers, each response contains only its own marker.
Negative/error path	Error-path testing	llama.cpp `test_security.py`	Invalid bearer ⇒ 401; malformed JSON ⇒ 4xx (not 5xx or hang); over-context ⇒ 4xx.

Table 2: Performance at 5 concurrent users (reproduced from prior study).

Engine	Latency (ms)	TTFT (ms)	Throughput (tok/s)	JSON (%)
vLLM	**260**	**260**	**61.5**	100
Ollama	1426	1426	12.1	100
llama-cpp-python	2343	2343	6.0	100

Table 3: New test suites and pass counts (unit mode).

Suite	Tests	Role
`test_contract.py`	9	Act 2 #1
`test_streaming_protocol.py`	7	Act 2 #2
`test_param_honoring.py`	11	Act 2 #3
`test_determinism.py`	3	Act 2 #4
`test_stat_regression.py`	5	Act 2 #5
`test_differential.py`	3	Act 2 #6
`test_concurrency_isolation.py`	3	Act 2 #7
`test_negative.py`	6	Act 2 #8
`test_properties.py`	8	Act 3 (Hypothesis)
`test_calibration.py`	3	Act 3 (calibration)
`test_stat_harness.py`	31	Act 3 (math pins)
All pre-existing suites	95	baseline
Total (unit mode)	**184**	—

7.3 Code coverage

Table 4 gives the line coverage reported by `pytest-cov`. The statistical harness is completely covered; the load runner is at 94%. Uncovered lines in `llm_tester.py` and `niah_runner.py` are the argparse entry points and the needle-in-a-haystack scenario, exercised only by integration runs.

Table 4: Line coverage by source module (unit mode).

Module	Stmts	Miss	Cov %
`stat_regression.py`	88	0	**100**
`comparator.py`	23	0	100
`ramp_runner.py`	37	0	100
`metrics.py`	78	2	97
`reporter.py`	78	4	95
`load_runner.py`	196	11	94
`config_loader.py`	52	3	94
`consistency.py`	55	5	91
`validator.py`	23	2	91
`html_reporter.py`	98	24	76
`niah_runner.py`	44	44	0
`llm_tester.py`	127	127	0
Total	**899**	**222**	**75**

7.4 Calibration

On the test host, `test_calibration.py` injects 150 ms of pre-first-chunk delay and 80 ms of inter-chunk delay; measured TTFT lands within ±25 ms and measured TPOT within ±25 ms (tolerance widened to 50 ms to absorb CI jitter, so the test also passes on loaded hosts).

7.5 Mutation testing

The run completes in ∼16 s and yields 247 killed, 29 survived, and 5 timeouts out of 281 mutations. The timeouts correspond to mutation-induced infinite loops in the rank-merging pass of `mann_whitney_u`; they are counted as killed by `mutmut`'s policy.

8 Discussion

The methodology inversion matters more than any individual number. Under the prior single-run comparison the vLLM advantage was a scalar; under the revised harness the same

advantage is an effect whose significance is quantified by a Mann–Whitney U test and whose measurement provenance is defended by the calibration and property layers. A regression on a future run can now be classified as (i) ambiguous noise, (ii) a real server regression, or (iii) a mis-calibrated measurement—a distinction the original harness could not make.

Three observations:

- Determinism at T=0 is not free. Backends that reorder floating-point reductions under concurrency violate it; our test detects this without assuming any particular reduction order.
- Concurrency isolation is the cheapest test with the highest bug-catching signal-to-noise. Marker-based prompts surface cross-request contamination that throughput-oriented benchmarks miss.
- The 16 percentage-point lift in mutation score was produced by a single small test class (10 pinned assertions); this suggests that most harnesses underpin their statistical claims with looser tests than the numbers deserve.

9 Framework Improvements Since Prior Report

- **SSE parser refactor.** A pure `parse_sse_stream` helper—taking an iterable of byte chunks and yielding `SSEEvent` objects—was extracted so the parser is independent of the HTTP transport and can be driven by property-based tests. A Hypothesis-discovered counter-example (`U+001E` as a line boundary under `str.splitlines`) was added to the regression suite.
- **Statistical regression harness.** Percentile, percentile-method bootstrap CI, and continuity-corrected Mann–Whitney U with tie correction, implemented against the standard library alone; no `scipy` dependency.
- **OpenAI schemas.** Draft-07 schemas for both `chat.completion` and `chat.completion.chunk` envelopes, used by the contract and protocol tests.
- **Parameter plumbing.** `max_tokens` and `stop` are threaded through the load runner so the metamorphic tests can vary them without patching internal state.
- **Baseline persistence.** `save_baseline`/ `load_baseline` store per-scenario JSON reports so statistical regressions can be compared across runs.

10 Limitations

The new tests are methodology-complete but not exhaustive: determinism is checked only at T=0; the differential suite uses three backends, not the full matrix; the mutation-testing target is currently only the statistical harness, not the load runner; and integration-mode runs depend on a backend listening at `localhost:8080`. Hardware and model choices from the prior study are unchanged and inherit its limitations.

11 Conclusion

The prior report answered *which backend is faster*. This revision answers *why we should believe the answer*. By layering eight named testing methodologies onto the same harness, and by testing the harness itself against calibration, property, and mutation baselines, the study now produces claims that are defensible as software-engineering artifacts rather than as single-run measurements. Under the evaluated conditions vLLM remains the best performer; the added machinery ensures that if that verdict ever changes, the change will be attributable and quantified.

12 Future Work

Future enhancements include: extending mutation testing to `load_runner.py`; adding P99 tail-latency regression tests; expanding the differential suite to the full model matrix; running the calibration harness periodically in CI to detect host drift; and integrating a GitHub Actions workflow that executes the unit suite on every PR and the integration suite nightly against a live backend.

References

[1] Thiruveedula, B., Arnold, K., and Franklin, R.J.M. *Benchmarking Local LLM Inference Servers Under Concurrent Load.* Class technical report, 2026.

[2] Kwon, W. et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention.* In *Proc. SOSP*, 2023.

[3] Gerganov, G. *llama.cpp: Port of LLaMA in C/C++.* https://github.com/ggerganov/llama.cpp.

[4] Ollama Team. *Ollama: Run Large Language Models Locally.* https://ollama.com.

[5] MacIver, D.R. et al. *Hypothesis: a new approach to property-based testing. J. Open Source Softw.*, 2019.

[6] Hovmöller, A. *mutmut: a mutation-testing system for Python.* https://github.com/boxed/mutmut.

[7] Hugging Face. *Text Generation Inference.* https://github.com/huggingface/text-generation-inference.